

Low-Level Design

RAG Customer Support Assistant

1. Modules and Functions

document_loader.py

```
load_pdf(path: str) → List[Document]
```

Uses PyPDFLoader. Call this once during ingestion.

chunker.py

```
chunk_docs(docs: List[Document]) → List[Document]
```

RecursiveCharacterTextSplitter with chunk_size=500, chunk_overlap=50.

embedder.py

```
get_embedder() → HuggingFaceEmbeddings
```

Returns the same embedding model for both ingestion and querying.

vector_store.py

```
build_store(chunks) / load_store() → Chroma
```

build_store runs once. load_store is called every time the app starts.

retriever.py

```
retrieve(query, store, k=3) → List[Document]
```

Calls store.similarity_search(query, k=k). Returns top-k chunks.

graph_nodes.py

```
6 node functions – see section 3
```

Each function takes state dict and returns a partial update dict.

workflow.py

```
build_graph() → CompiledGraph
```

Assembles and compiles the LangGraph StateGraph.

main.py

```
main() → None
```

CLI loop. Loads store and graph, then runs graph.invoke() per query.

2. Graph State

All nodes share one state dictionary (TypedDict):

```
class GraphState(TypedDict):
    query:      str          # user's question
    store:      Any          # ChromaDB instance
    chunks:     list         # retrieved Document chunks
    answer:     str          # final answer
    route:      str          # 'llm' or 'hitl'
    escalated:  bool         # True if human handled it
```

3. Node Responsibilities

input_node

Does nothing. The query is already set in state before invoke().

retrieve_node

Calls `retrieve(state['query'], state['store'])`. Sets `state['chunks']`.

route_node

Sets `state['route']` to 'hitl' if `chunks` is empty or 'human' is in query. Otherwise 'llm'.

llm_node

Joins chunk texts into a context string. Sends prompt to LLM. Sets `state['answer']`.

hitl_node

Prints the query, pauses with `input()`, stores the typed answer in `state['answer']`.

output_node

No changes to state. `main.py` reads `state['answer']` after `invoke()` returns.

4. Graph Edges

```
input → retrieve → route
route → llm      (when route == 'llm')
route → hitl     (when route == 'hitl')
llm  → output
hitl → output
output → END
```

5. Routing Logic

```
def route_node(state):
    no_context = len(state['chunks']) == 0
```

```
wants_human = 'human' in state['query'].lower()
route = 'hitl' if (no_context or wants_human) else 'llm'
return { 'route': route, 'escalated': route == 'hitl' }
```

Two rules only: escalate if there is nothing to retrieve, or if the user explicitly asks for a human.

6. HITL Node

```
def hitl_node(state):
    print(f'[HITL] Query: {state["query"]}')
    print('[HITL] Agent, please type the answer:')
    answer = input('Agent > ')
    return { 'answer': answer }
```

- Triggered when route == 'hitl'
- Human types the answer in the terminal
- Answer flows into output_node exactly like an LLM answer

7. Error Handling

PDF not found

load_pdf() raises FileNotFoundError → caught in main, print message and exit.

No chunks retrieved

retrieve() returns [] → route_node sets route='hitl' automatically.

LLM API failure

Wrap llm.invoke() in try/except → on error, set route='hitl' and escalate.

Empty user input

Check in main loop: if not query.strip(): continue.